

Tabăra de pregătire a lotului național de informatică juniori

Bistrița, 10-15 mai 2026

Descrierea soluțiilor Baraj 3

Comisia științifică

12 mai 2026

Problema 1. Cleanup

Propusă de: prof. Zoltan Szabó, Inspectoratul Școlar Județean Mureș

Înainte de a ne apuca să rezolvăm problema, este foarte util să observăm că atingerea unui câmp de coordonate (i, j) are efect asupra câmpurilor $A_{i-1,j}$, $A_{i,j+1}$, $A_{i+1,j}$ și $A_{i,j-1}$. Toate aceste câmpuri au paritate diferită față de paritatea câmpului pe care l-am atins.

În consecință orice combinație de atingeri ale câmpurilor de poziție pară afectează exclusiv câmpuri de poziție impară, și invers, orice combinație de atingeri ale câmpurilor de poziție impară afectează exclusiv câmpuri de poziție pară.

Deci problema se descompune în două subprobleme disjuncte, iar soluția se obține din combinarea celor două rezultate: în cazul $C = 1$, numărul soluțiilor se obține prin înmulțire, iar în cazul $C = 2$, numărul minim de atingeri se obține prin adunare.

Găsirea rezultatului optim se realizează prin generarea tuturor cazurilor, și memorarea soluției optime.

Cerința 1 (numărul soluțiilor distincte)

Prin scrierea unui algoritm backtracking, putem deduce prin observare directă că numărul de soluții distincte este 2^N . Întrucât avem $N \leq 40$, acest număr se încadrează în tipul `long long`. Să demonstrăm această formulă.

Să demonstrăm întâi că există 2^N soluții pentru o tablă **inițial stinsă**. Iată o demonstrație vizuală pentru $N = 7$. Valoarea finală a unui câmp este xor-ul atingerilor vecinilor săi. De aceea, fie $b_1, b_2, \dots, b_7$ o combinație oarecare de atingeri (valori 0/1) pentru prima linie. Atingerile pentru restul matricei sunt impuse. De exemplu, atingerea de la câmpul $(2, 2)$ trebuie să anuleze efectele atingerilor de la câmpurile $(1, 1)$ și $(1, 3)$. Rezultă matricea:

b_1	b_2	b_3	b_4	b_5	b_6	b_7
b_2	$b_1 \oplus b_3$	$b_2 \oplus b_4$	$b_3 \oplus b_5$	$b_4 \oplus b_6$	$b_5 \oplus b_7$	b_6
b_3	$b_2 \oplus b_4$	$b_1 \oplus b_3 \oplus b_5$	$b_2 \oplus b_4 \oplus b_6$	$b_3 \oplus b_5 \oplus b_7$	$b_4 \oplus b_6$	b_5
b_4	$b_3 \oplus b_5$	$b_2 \oplus b_4 \oplus b_6$	$b_1 \oplus b_3 \oplus b_5 \oplus b_7$	$b_2 \oplus b_4 \oplus b_6$	$b_3 \oplus b_5$	b_4
b_5	$b_4 \oplus b_6$	$b_3 \oplus b_5 \oplus b_7$	$b_2 \oplus b_4 \oplus b_6$	$b_1 \oplus b_3 \oplus b_5$	$b_2 \oplus b_4$	b_3
b_6	$b_5 \oplus b_7$	$b_4 \oplus b_6$	$b_3 \oplus b_5$	$b_2 \oplus b_4$	$b_1 \oplus b_3$	b_2
b_7	b_6	b_5	b_4	b_3	b_2	b_1

Tabela 1: Alegerea unor atingeri pentru prima linie impune restul atingerilor. Simbolul $\oplus$ desemnează operația sau exclusiv (xor).

Algoritmul garantează stingerea tuturor liniilor de la 1 la $N - 1$. Întrebarea este cum garantăm că și ultima linie de câmpuri se va stinge. Să observăm că matricea are simetrie la rotație cu 180° . De exemplu, câmpul $(7, 6)$ va avea starea finală $b_3 \oplus (b_1 \oplus b_3) \oplus b_1$, identică cu starea câmpului $(1, 2)$. Similar, stările pe ultima linie corespund cu cele de pe prima linie (în ordine inversă), pentru care algoritmul a garantat stingerea.

Întrucât alegerea valorilor b_i a fost arbitrară, rezultă că, pentru o tablă inițial stinsă, există 2^N soluții. Să tratăm acum cazul general, în care tabla inițială nu este neapărat stinsă. Enunțul garantează existența unei soluții S . Combinarea soluției S cu oricare dintre cele 2^N soluții pentru tabla stinsă, demonstrate anterior, va conduce tot la o tablă stinsă. Acest lucru este ușor de demonstrat: pornind de la tabla inițială, aplicăm atingerile din S pentru a stinge tabla, apoi atingerile din soluția aleasă pentru tabla stinsă. Dacă ambele soluții ating același câmp câte o dată, eliminăm acele atingeri din soluția combinată, conform observațiilor din enunț.

În concluzie, câtă vreme datele de intrare admit o soluție, ele admit exact 2^N soluții.

Cerința 2 (generarea unei soluții cu număr minim de atingeri)

O soluție brută (41 de puncte din 77 posibile)

Un algoritm greedy foarte simplu parcurge toate liniile de la 2 la N și pentru fiecare poziție de coordonate (i, j) , atingerea acestui câmp este unic determinată de valoarea lui $A_{i-1,j}$, așa cum a fost ea modificată de atingerile anterioare. Astfel putem genera în complexitate $O(N^2)$ o soluție corectă, dar nu cu număr minim de pași.

Soluție backtracking pe diagonale (67 de puncte din 77 posibile)

Vom efectua atingerile câmpurilor de sus în jos, de la stânga spre dreapta, pe linii oblice paralele cu diagonala secundară. Se observă că pentru primele N linii oblice, se poate alege ca și primă atingere atât 0 cât și 1, iar celelalte câmpuri devin unic determinate în funcție de celelalte valori deja determinate.

Următoarele $N - 1$ linii oblice vor fi unic determinate de valorile existente în acest moment. Întrucât sunt N linii oblice, fiecare cu câte 2 valori de pornire, și alte $N - 1$ linii unic determinate, rezultă că pentru cazul $C = 1$ avem 2^N soluții distincte.

Complexitatea algoritmului de mai sus este $O(2^N \cdot N^2)$ și nu vom obține 77 de puncte. Totuși, algoritmul funcționează pentru valori $N \leq 22$ și va obține aproximativ 41 de puncte.

Dar, aplicând ideea separat pe cele două subprobleme de parități diferite, vom obține de două ori complexitatea $O(2^{N/2} \cdot N^2)$, și combinând cele două rezultate, vom rezolva problema în $O(2^{N/2} \cdot N^2)$.

Soluție backtracking pe linii (77 de puncte din 77 posibile)

Generând toate cele 2^N configurații de atingeri posibile pentru prima linie, restul liniilor, pentru fiecare configurație găsită, conțin atingeri unic determinate prin algoritmul greedy descris în secțiunea anterioară, de complexitate $O(N^2)$. Concret, când evaluăm câmpul (i, j) , el este ultimul care mai poate modifica starea câmpului $A_{i-1,j}$, întrucât am evaluat deja ceilalți vecini ai săi. Pentru a obține $A_{i-1,j} = 0$, atingerea sau neatingerea câmpului (i, j) este impusă.

Deci această abordare va avea complexitatea $O(2^N \cdot N^2)$. Aplicând ideea separat pe cele două subprobleme de parități diferite, vom obține de două ori complexitatea $O(2^{N/2} \cdot N^2)$, și combinând cele două rezultate, vom rezolva problema în $O(2^{N/2} \cdot N^2)$, care obține 67 de puncte dacă operăm pe o matrice.

Pentru punctajul întreg, observăm că liniile matricei sunt formate din cifre 0/1, și le putem interpreta ca numere întregi de tip `long long`. Așadar vom folosi un vector A de N valori `long long`, unde $A[i]$ conține N biți. Atingerile le putem considera operații xor pe biți, ceea ce permite prelucrarea datelor în paralel.

Dorim să aflăm, pentru câmpul (i, j) , dacă el va mai trebui modificat prin atingerea câmpului $(i + 1, j)$. Aceasta depinde de valoarea câmpurilor de sus, stânga și dreapta și putem calcula rezultatul simultan pentru toate coloanele j cu formula $A[i] \wedge (A[i] \ll 1) \wedge (A[i] \gg 1)$.

Astfel, putem calcula fiecare linie din anterioara în $O(1)$, ceea ce duce la complexitatea totală $O(2^{N/2} \cdot N)$.

Problema 2. Lac

Propusă de: prof. Gheorghe-Eugen Nodda, Centrul Județean de Excelență Gorj

Considerații preliminare

Se consideră n brazi dispuși circular. Pentru fiecare poziție i ($1 \leq i \leq n$) se cunosc înălțimile brazilor H_i și înălțimile reflexiilor acestora în lac Hr_i . Dublăm vectorii H și Hr pentru a evita tratarea specială a secvențelor circulare.

La momentul t , reflexia devine:

$$R_i(t) = \max(Hr_i - t, 0)$$

Un dreptunghi valid trebuie să fie înscris în ambele zone (atât în cea a brazilor, cât și în cea a reflexiilor).

Observații:

- $\min(Hr - t) = \min(Hr) - t$
- $\min(R) > 0$, pentru ca o secvență să fie validă

Dorim să găsim o secvență circulară de lungime k pentru care aria maximă a dreptunghiului este:

$$A = k \cdot (\min(H) + \min(R))$$

cu condiția $\min(R) > 0$. Rezultă că lungimea secvenței k trebuie să fie un divizor al lui A , iar:

$$t = \min(Hr) + \min(H) - \frac{A}{k}$$

Subtask-uri și Optimizări

Subtask 1 - Soluție brută $O(n^3 \cdot t)$

Se parcurg toate valorile posibile pentru t și toate secvențele. Minimele se calculează prin parcurgere completă la fiecare pas.

Dacă minimele sunt menținute incremental la extinderea secvenței, se reduce complexitatea determinării acestora la $O(n^2 \cdot t)$.

Subtask 2 - Divizori și ferestre glisante $O(n \cdot \sqrt{A} \cdot t)$

Întrucât k divide A , calculăm toți divizorii lui A . Parcurgem toate secvențele valide de lungime k (cu algoritmul clasic de minim în fereastră glisantă) și determinăm minimele eficient folosind două structuri de tip **deque**.

Optimizare - eliminarea variabilei t , $O(n \cdot \sqrt{A})$

Pentru fiecare secvență calculăm direct:

$$t = \min(Hr) + \min(H) - \frac{A}{k}$$

și verificăm condițiile de validitate:

$$t \geq 0 \quad \text{și} \quad \min(Hr) > t.$$

Pentru obținerea punctajului maxim este necesară eliminarea redundanțelor în procesarea secvențelor. Pentru o lungime a ferestrei k și lățimea corespunzătoare A/k , observăm că nu are sens să facem efortul $O(n)$ de glisare a ferestrei în două cazuri:

1. A/k este mai mic decât înălțimea minimă a oricărui brad. Atunci în mod sigur dreptunghiul va avea înălțime mai mare decât A/k .
2. A/k este mai mare decât suma maximă a înălțimii oricărui brad și a oricărei reflexii. La orice moment $t \geq 0$, valoarea A/k va fi prea mare.

Dacă notăm:

- $H_{\min}$ - valoarea minimă a înălțimilor H_i
- $H_{\max}$ - valoarea maximă a înălțimilor H_i

- $R_{\max}$ - valoarea maximă a înălțimilor Hr_i

Putem restrânge valorile posibile pentru k folosind observații:

$$\frac{A}{k} \leq H_{\max} + R_{\max} \Rightarrow k \geq \frac{A}{H_{\max} + R_{\max}}$$

$$\frac{A}{k} > H_{\min} \Rightarrow k < \frac{A}{H_{\min}}$$

Prin urmare, analizăm doar valorile:

$$k \in [left, right]$$

unde:

$$left = \left\lceil \frac{A}{H_{\max} + R_{\max}} \right\rceil, \quad right = \left\lfloor \frac{A - 1}{H_{\min}} \right\rfloor$$

Preprocesare minime $O(n \log n)$

Ca alternativă la algoritmul de minim în fereastră glisantă, putem precălcule minimele oricărei subsecvențe cu **RMQ** (*Sparse Table*). Astfel, calculul minimelor pentru orice secvență de lungime k se face în $O(1)$. Complexitatea evaluării unei lungimi k rămâne $O(n)$, dar în practică se comportă mai bine.

Problema 3. Tactica

Propusă de: prof. Marius Nicoli, Colegiul Național Frații Buzești Craiova

Subtask 1. $x_N - x_1 \leq L$

Când toți jucătorii sunt deja grupați într-o fereastră de lungime cel mult L , atunci ei trebuie să se disperseze pentru a ocupa lungimea L . De aceea, răspunsul va fi $L - (x_N - x_1)$.

Subtask 2. $N < 100$

Deoarece enunțul permite 100 de apeluri pentru punctaj maxim, putem apela funcția `getX()` de N ori pentru a afla toate coordonatele. Apoi căutăm poziția optimă a ferestrei. Iată o abordare în $O(N)$.

Pentru orice amplasare a ferestrei care nu conține jucători la capete, există o altă amplasare, de cost mai mic sau egal, în care la unul dintre capetele ferestrei se află un jucător. Pentru demonstrație, fie o amplasare a ferestrei care nu are jucători la niciun capăt. Fie s și d numerele de jucători aflați strict în stânga și respectiv strict în dreapta ferestrei. Iau naștere trei posibilități:

- Dacă $s < d$, atunci merită să deplasăm fereastra la dreapta cu o poziție. Costurile celor s jucători vor crește cu 1, dar costurile celor d jucători vor scădea cu 1, deci costul total va scădea.

- Dacă $s > d$, atunci printr-un raționament similar merită să deplasăm fereastra la stânga cu o poziție.
- dacă $s = d$, putem deplasa fereastra în orice direcție cu o poziție, iar costul total va rămâne constant.

Prin urmare, este suficient să calculăm suma deplasărilor în (cel mult) $2N$ amplasări ale ferestrei. Pe măsură ce deplasăm fereastra de la o poziție la următoarea, menținem numărul de jucători și suma coordonatelor pentru partea stângă, respectiv partea dreaptă.

Subtask 3, implementare în $8 \log MAX_X$

Fie $C(x)$ costul regroupării în fereastra $[x, x + L]$. Intuiția ne spune că, dacă $C(x_M)$ este punctul în care $C(x)$ își atinge minimul, atunci cu cât ne depărtăm de x_M , fie în stânga, fie în dreapta, cu atât $C(x)$ va crește. Să demonstrăm formal această intuiție.

Pentru fereastra $[x_M, x_M + L]$ vom avea, conform argumentului de mai sus, că $s = d$, cu alte cuvinte sunt la fel de mulți jucători în stânga ferestrei ca și în dreapta. (Pentru completitudine, se poate ca $|s - d| = 1$ în cazuri-limită, dar argumentul nu se modifică.) Acum să considerăm că deplasăm fereastra pas cu pas spre stânga. Costul jucătorilor din stânga va scădea cu s la fiecare pas, iar costul jucătorilor din dreapta va crește cu d la fiecare pas. Câtă vreme $s = d$, suma costurilor, adică $C(x)$, va rămâne la valoarea minimă. Dar, pe măsură ce fereastra „trece” peste jucători, pot surveni evenimentele:

1. Un jucător din stânga intră în fereastră, deci s scade cu 1.
2. Un jucător iese din fereastră la dreapta, deci d crește cu 1.

În orice caz, diferența $d - s$, inițial 0, va deveni pozitivă și va căpăta valori tot mai mari. La fiecare scădere cu 1 a lui x , valoarea lui $C(x)$ va crește cu această diferență $d - s$, deci se va abate monoton de la minimul $C(x_M)$. Pentru deplasarea la dreapta argumentul este simetric.

Astfel am demonstrat că secvența $C(x)$ este bitonă. Putem aplica algoritmul clasic de căutare a minimului. La fiecare iterație, dacă m este coordonata mijlocului intervalului de înjumătățit, atunci este nevoie să testăm două coordonate, de exemplu $m - 1$ și m .

Odată aleasă o fereastră de testat, $[x, x + L]$, cum îl aflăm pe $C(x)$? Toți jucătorii din stânga ferestrei trebuie să se deplaseze până la x . Suma acestor deplasări este

$$(x - x_1) + (x - x_2) + \dots + (x - x_s) = sx - \sum_{i=1}^s x_i$$

Exprimarea echivalentă în cod este `getNumber(1, x - 1) * x - getSumX(1, x - 1)`. Similar procedăm și pentru zona din dreapta ferestrei.

Numărul total de apeluri este $8 \log MAX_X$, care provin din:

- $\log N$ iterații pentru a determina exact valoarea lui x_M ;
- două puncte de testat pentru mijlocul ales ($m - 1$ și m);

- două interogări `getNumber()` și două interogări `getSumX()` pentru fiecare dintre aceste două puncte.

Această implementare poate face 240 de interogări și obține 0,6 din punctajul maxim. O eventuală reducere la $4 \log MAX_X$ (120 de apeluri) obține 0,8 din punctajul maxim.

Subtask 3, implementare în $2 \log MAX_X$

Odată ce am demonstrat că regruparea optimă are $s = d$, putem căuta binar capătul stâng al ferestrei. Așadar, pentru mijlocul curent, m , calculăm

- `s = getNumber(1, m - 1);`
- `d = getNumber(m + 1 + 1, MAX_X);`

Apoi restrângem căutarea la jumătatea cu valoarea mai mare. Astfel facem doar două apeluri per înjumătățire și obținem $2 \log MAX_X$ apeluri, plus încă 4 apeluri la final pentru a calcula costul regrupării (circa 64 de apeluri, suficient pentru punctaj maxim).

Subtask 3, implementare în $2 \log N$

Iată o idee și mai eficientă, independentă de coordonata maximă. Dacă știm că vom lăsa la fel de mulți jucători în stânga ferestrei ca și în dreapta, înseamnă că intervalul jucătorilor din fereastră va fi centrat pe $x_{N/2}$. Atunci căutăm binar valoarea minimă al lui k pentru care intervalul $[x_k, x_{N+1-k}]$ are lungime cel mult egală cu L . Apoi extindem acest interval până la lungimea L și îi calculăm costul.

Deoarece facem căutarea binară printre cele 10^6 coordonate ale jucătorilor, nu printre cele 10^9 coordonate posibile, acest algoritm face doar $2 \log N + 4$ apeluri (circa 44).

Echipa

Problemele pentru această etapă au fost pregătite de:

- prof. Emanuela Cerchez, Colegiul Național „Emil Racoviță” Iași
- prof. Ionel-Vasile Piț-Rada, Colegiul Național Traian, Drobeta Turnu Severin
- șef lucrări dr. Mihai Nan, Facultatea de Automatică și Calculatoare, Universitatea Națională de Știință și Tehnologie Politehnica București
- prof. Dorin-Mircea Rotar, Colegiul Național „Samuil Vulcan” Beiuș
- prof. Mihai Bunget, Colegiul Național „Tudor Vladimirescu” Târgu-Jiu
- prof. Ciprian Cheșcă, Liceul Tehnologic Grigore Moisil Buzău
- prof. Gheorghe-Eugen Nodea, Centrul Județean de Excelență Gorj

- prof. Petru Simion Oprea, Liceul Regina Maria Dorohoi
- prof. Zoltan Szabó, Inspectoratul Școlar Județean Mureș
- prof. Marius Nicoli, Colegiul Național Frații Buzești Craiova
- stud. Alin Gabriel Răileanu, Facultatea de Informatică, Universitatea „Al. I. Cuza” Iași
- instr. Cătălin Frâncu, Nerdvana 2.0 București
- stud. Aureliu Valentin Antonie, Facultatea de Automatică și Calculatoare, Universitatea Națională de Știință și Tehnologie Politehnica București
- stud. Ioan-Cristian Pop, Facultatea de Automatică și Calculatoare, Universitatea Națională de Știință și Tehnologie Politehnica București
- prof. Dana Lica, Centrul Județean de Excelență Prahova
- prof. Bogdan-Ioan Popa, Liceul Teoretic Just4Kids București
- stud. Cristian Luchian, Facultatea de Informatică, Universitatea „Al. I. Cuza” Iași
- stud. Andrei Boacă, Facultatea de Informatică, Universitatea „Al. I. Cuza” Iași
- stud. Rareș-Andrei Cotoi, Facultatea de Matematică și Informatică, Universitatea „Babeș-Bolyai” Cluj-Napoca
- stud. Lucian-Andrei Badea, TU Delft
- prof. Marinel Șerban, Centrul Județean de Excelență Iași